1 Unit-Tests für Lexer, Parser und Codegenerator

Unit-Tests überprüfen gezielt einzelne Einheiten, wie Funktionen und Komponenten einer Software (Meszaros 2007). Sie sind besonders hilfreich, wenn sie auf eine bestimmte Phase des Compilationsprozesses abzielen, da dadurch präziser die Ursache eines Fehlers identifiziert werden kann.

1.1 Lexer

```
EXPECT_TRUE(Lex("#include <\\\\\\n").empty());
```

1.2 Parser

1.3 Codegenerator

2 Differential Testing

Ein Programm sollte bei gleichem Quellcode auch mit unterschiedlichen Compilern und Compileroptionen das gleiche beobachtbare Verhalten zeigen. Voraussetzung ist, dass der Quellcode kein undefiniertes Verhalten enthält und relevante Unterschiede zwischen den Zielplattformen berücksichtigt werden.

Dabei werden folgende Varianten unterschieden:

- Cross-Compiler Testing (Terres und Feitosa 2025)
- Cross-Version Testing (Terres und Feitosa 2025)
- Cross-Optimization Testing (Terres und Feitosa 2025)
- Cross-Architecture Testing (Elnaggar, Delgado und Fung 2025)

2.1 Cross-Compiler Testing

Unterschiedliche Compiler, wie beispielsweise GCC und Clang, arbeiten intern unterschiedlich, müssen aber die Semantik eines Programms bei der Übersetzung erhalten.

2.2 Cross-Version Testing

Verschiedene Versionen eines Compilers werden miteinander verglichen, um Regressionen aufzudecken. Bei unveränderten Sprachregeln und vergleichbaren Compileroptionen sollte das beobachtbare Programmverhalten erhalten bleiben.

2.3 Cross-Optimization Testing

Compiler bieten unterschiedliche Optimierungsstufen an, beispielsweise `-O1`, `-O2` und `-O3`. Dabei können unter anderem Kontrollstrukturen verändert und Ausdrücke vereinfacht werden. Das definierte beobachtbare Programmverhalten muss dabei erhalten bleiben.

2.4 Cross-Architecture Testing

Das Programmverhalten wird auf unterschiedlichen Zielarchitekturen verglichen, beispielsweise x86, ARM, MIPS und RISC-V. Dabei müssen plattformabhängige Eigenschaften wie die Größe von Datentypen und die Byte-Reihenfolge berücksichtigt werden.

Abweichende Ergebnisse können auf Compilerfehler hinweisen. Der Vergleich allein zeigt jedoch weder eindeutig, *welche* Konfiguration fehlerhaft ist, noch, *warum* der Fehler auftritt.

Csmith (Csmith Project 2026) generiert zufällige Programme in C. Diese können mit verschiedenen Compilern und Compileroptionen getestet werden. Die Ergebnisse werden anschließend verglichen, um mögliche Fehler aufzudecken.

```
csmith > random2.c
gcc random2.c -I$HOME/csmith/include -o random2_gcc
clang random2.c -I$HOME/csmith/include -o random2_clang
./random2_gcc > gcc_output.txt
./random2_clang > clang_output.txt
diff -u gcc_output.txt clang_output.txt
```

Der obige Ausschnitt zeigt Befehle für eine Unix-Shell. Ein zufälliges C-Programm wird generiert, mit GCC und Clang kompiliert und anschließend ausgeführt. Die Ausgaben werden in Textdateien gespeichert und mit `diff` verglichen. Der Include-Pfad muss zur lokalen Csmith-Installation passen.

Die erzeugten Binärdateien dürfen sich unterscheiden, da die Compiler intern unterschiedlich arbeiten. Unter vergleichbaren Bedingungen muss das definierte beobachtbare Verhalten und damit auch die verglichene Ausgabe übereinstimmen.

3 Gute Compiler-Fehlermeldungen

Hilfreiche Compilerfehlermeldungen erleichtern es, einen Fehler zu lokalisieren und zu beheben. Die folgende Tabelle enthält illustrative Beispiele; der genaue Wortlaut hängt unter anderem von der Compilerversion und dem Quellcode ab.

Tabelle 1: Illustrative Beispiele für Compilerfehlermeldungen

Angabe	**Beispiel**
Schweregrad	`Error/Warn/Info/Critical`
Fehlerart	`Syntax Error`
Kontext	`In der Funktion main()`
Betroffene Position	`main.py, Zeile 10, Spalte 3`
Betroffener Quelltext	`print 1`
Vorschlag zur Behebung	`Füge Klammern hinzu`

Um sicherzustellen, dass Compilerfehlermeldungen stets konsistent und hilfreich sind, können auch diese mit Unit-Tests überprüft werden.

Literatur

Csmith Project (2026). *Csmith, a random generator of C programs*. URL: https://github.com/csmith-project/csmith/blob/master/README.md (besucht am 30. 10. 2026).

Elnaggar, Rana, Brian Delgado und Jason M. Fung (11. Apr. 2025). *RAG-Based Fuzzing of Cross-Architecture Compilers*. Version 1. arXiv: 2504.08967 [cs.CR]. URL: https://arxiv.org/abs/2504.08967v1 (besucht am 30. 10. 2026).

Meszaros, Gerard (2007). *xUnit Test Patterns: Refactoring Test Code*. Boston: Addison-Wesley. ISBN: 9780131495050.

Terres, Luiz Paulo Grafetti und Samuel da Silva Feitosa (29. Okt. 2025). „Differential Fuzzing Go Compilers using LLMs: A Methodological Proposal“. In: *Anais da Escola Regional de Engenharia de Software (ERES)*. Sociedade Brasileira de Computação, S. 50–59. DOI: 10.5753/eres.2025.16857. URL: https://sol.sbc.org.br/index.php/eres/article/view/40378 (besucht am 30. 10. 2026).